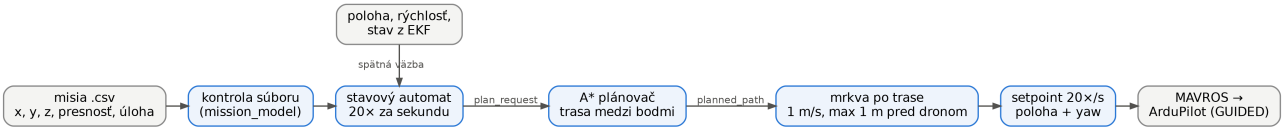
A1.2 — Indoor mission in the hangar (waypoints, tasks, A*)

A simulated drone (ArduCopter SITL + Gazebo, through MAVROS) flies a mission from a file: it takes off, has the A1.1 A* planner plan a route to every waypoint, flies it nose-first, performs the waypoint's task and at the end lands and disarms. Everything is driven by the `mission_executor_node` and its state machine.

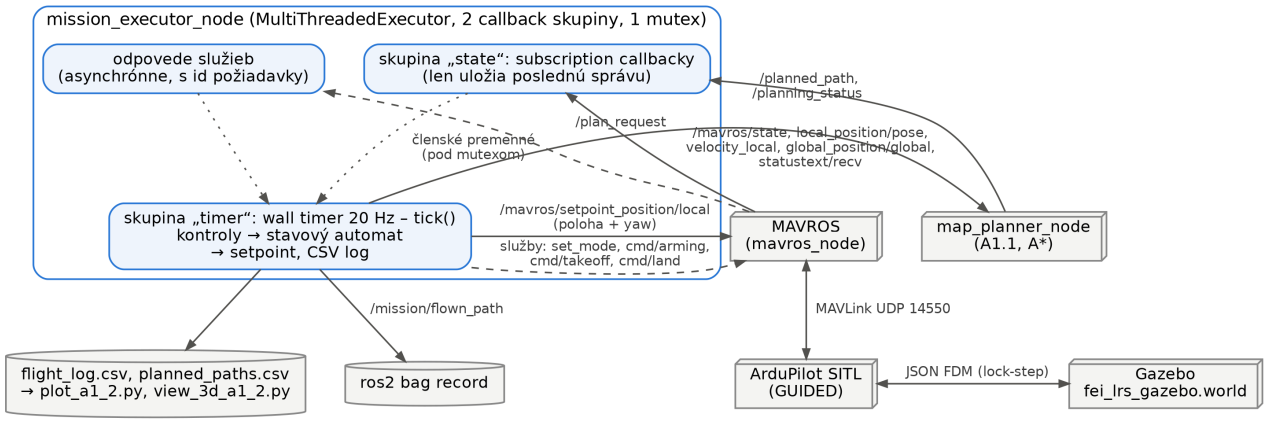
Run: `./run_a1_2.sh [mission.csv]` (default `missions/hangar_example.csv`), more in README.md.
Recordings: `results/A1_2/`.

Diagrams

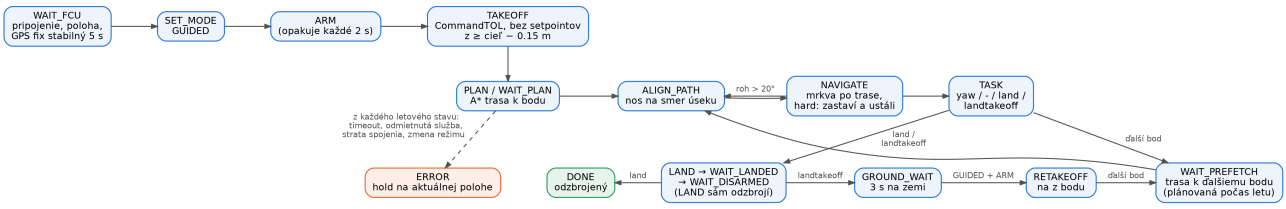
Data flow: the mission contains only goal points; the route between them is computed by A* and the drone follows it with a "carrot".



ROS diagram: two nodes (planner, executor). The executor runs in a `MultiThreadedExecutor` with two callback groups (the `mt_executor_demo` pattern): subscriptions only store the latest message, a 20 Hz timer makes every decision. Shared data is protected by one mutex.



State machine: the dashed arrow (ERROR) applies from every flight state.



Decisions

Mission from a file

What: the mission path is the ROS parameter `mission_file`, format `x,y,z,precision,task` (`hard`/`soft`; task `-`, `takeoff`, `land`, `landtakeoff`, `yaw<angle>`), `#` is a comment. A wrong field count, a non-number or an unknown task stops the node on the ground with the line number.

Why: at the defence we get an unknown file; a typo must end with a clear error, not a strange flight.

Frames and the yaw convention

What	Frame
mission, planner, plots (`map`)	map and Gazebo frame: x = north, y = west, z up
MAVROS local	ENU (x = east, y = north), rotated by 90° from `map`
inside ArduPilot	NED, the ENU ↔ NED conversion is done by MAVROS

`map ↔ local` is a 90° rotation about z (`map_to_local_yaw_offset_deg`), verified: the drone starts in Gazebo at (12.99, 7.00), MAVROS reports (–7.00, 12.99).

`yaw<angle>` is an **absolute** heading in the map frame, in degrees, **positive counter-clockwise**, 0° = map +x. `yaw90` = nose to +y, `yaw-45` = 45° clockwise.

Start-up: readiness, GUIDED, arming, takeoff

What: in `WAIT_FCU` we wait for the connection, a local position and a stable GPS fix (5 s). Services (set_mode, arming, takeoff, land) are called asynchronously, the response is checked and we only continue once `/mavros/state` confirms the real state; no response within 5 s → ERROR. After arming we send `NAV_TAKEOFF` and **no position setpoints** until the drone is within 0.15 m of the target height. If it disarms on the ground we repeat GUIDED → ARM → TAKEOFF (at most 3×).

Why: a sent request is not an accepted one. A position setpoint in GUIDED cancels a takeoff in progress and the drone disarms itself after 10 s on the ground (the original "arms but never takes off" bug). ArduPilot messages (`PreArm: ...`) are copied into our log, so the reason for a refusal is visible.

Planning and following the route ("carrot")

What: for every waypoint we send `plan_request` [position, waypoint]; while flying we already plan the next leg. A route is rejected if the planner moved the goal further than the waypoint tolerance. The setpoint (carrot) slides along the straight segment at 1 m/s and is never more than 1 m ahead of the drone. Intermediate route points count as passed within 0.45 m (`pass_radius`), without stopping.

Why: the A* route is collision-free only on the straight line. When we sent only the segment end, ArduPilot flew with different horizontal and vertical limits and left the line by up to 0.56 m (e.g. descending early over a rack). Pre-planning removes the wait for A* at waypoints.

Pros / cons: + the drone stays on the route and does not stop at intermediate points. – Speed is limited to 1 m/s.

Heading

What: between waypoints the drone flies nose along the segment (`atan2(Δy, Δx)`). Before a route and

at corners sharper than 20° it first turns on the spot (within 5° for 0.5 s); gentler bends are flown smoothly. The yaw command is continuous (unwrapped): from 170° to −170° the drone turns by 20°, not by 340°.

Hard and soft precision

	Tolerance	When the waypoint is reached
`hard`	0.15 m	within 0.15 m **and** speed ≤ 0.2 m/s continuously for 1 s
`soft`	0.55 m	within 0.55 m, no stop

Why: 0.15 m is in the recommended 10–20 cm range and is reached reliably in SITL (measured 2.9–9.7 cm). At 0.55 m the drone passes the point smoothly. The "stable for 1 s" condition rules out a fly-by that is inside the tolerance for a single sample. Distance from obstacles is not the job of the tolerance but of the planner's `safety_radius`.

Yaw task

What: at the waypoint the target yaw is set to the given angle; done when the nose is within 5° of it continuously for 0.5 s; the position is held. The angle is sent in the position setpoint, which GUIDED latches (a velocity command would expire after `GUID_TIMEOUT`).

Land and landtakeoff

What: `cmd/land` switches to LAND. After touchdown (z ≤ 0.15 m, speed ≤ 0.15 m/s) we wait for LAND to **disarm the drone by itself**; our own disarm is sent only after 5 s. For `landtakeoff` we then wait 3 s, switch to GUIDED again, arm and take off to the waypoint height.

Why: ArduPilot accepts a disarm only once its own land detector confirms touchdown. The original code treated the refusal as an error and the mission ended at the landtakeoff waypoint.

Errors and safety

- **ERROR:** state timeout, rejected service, lost connection, position older than 5 s, unexpected disarm → the drone gets its current position as the target and hovers.
- If someone switches the mode away from GUIDED in flight, the mission stops (we do not fight the pilot).
- **Slow simulation** (Gazebo at 0.2–0.3× real time): the script measures the factor, multiplies the executor's timeouts and divides the carrot speed.
- **Collision check after the flight:** `plot_a1_2.py` computes, for every pose, the distance to the nearest obstacle (walls, pillars, roof, racks; not the floor) and compares it with the drone radius of 0.33 m.

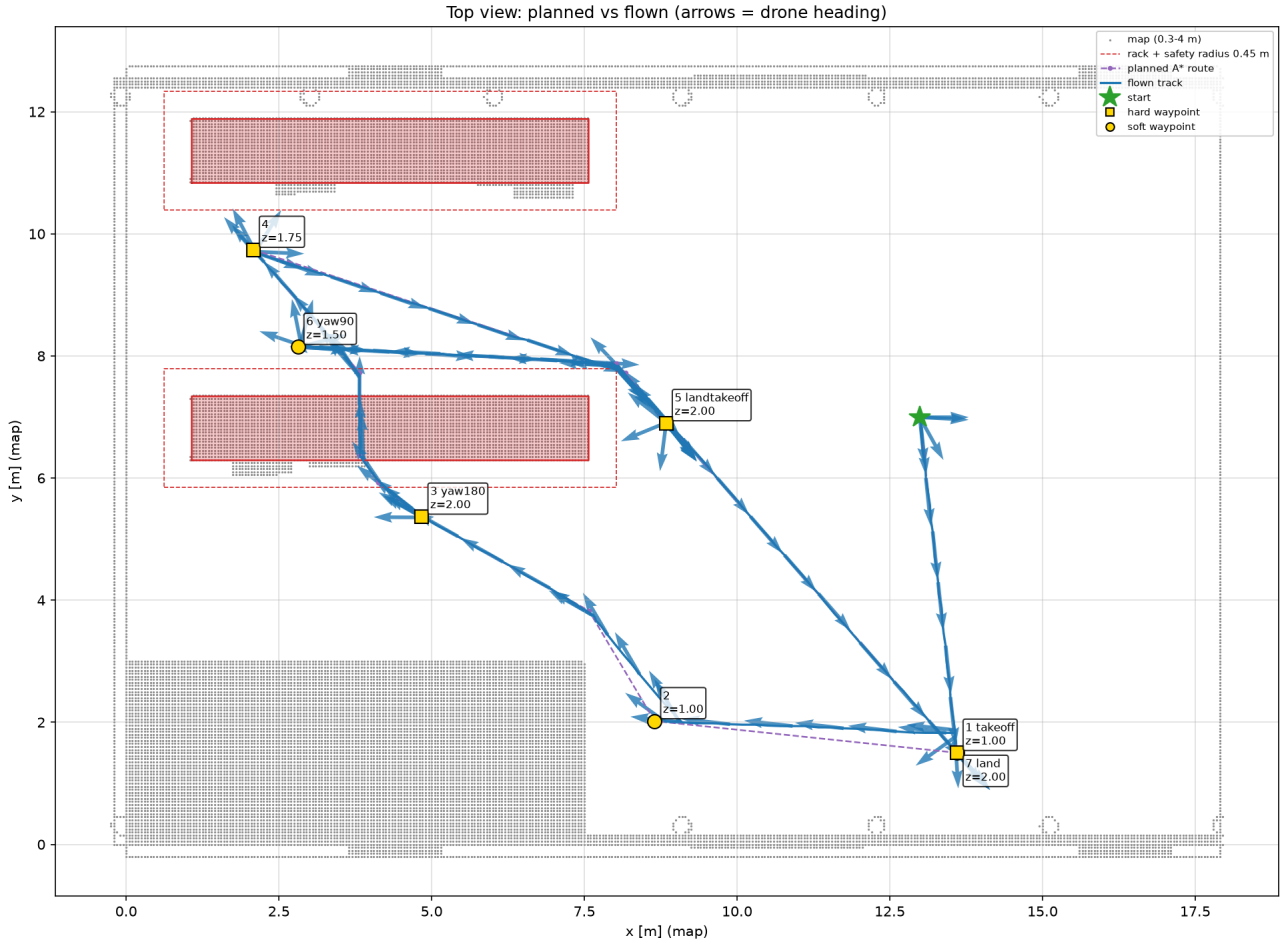
Results (simulation)

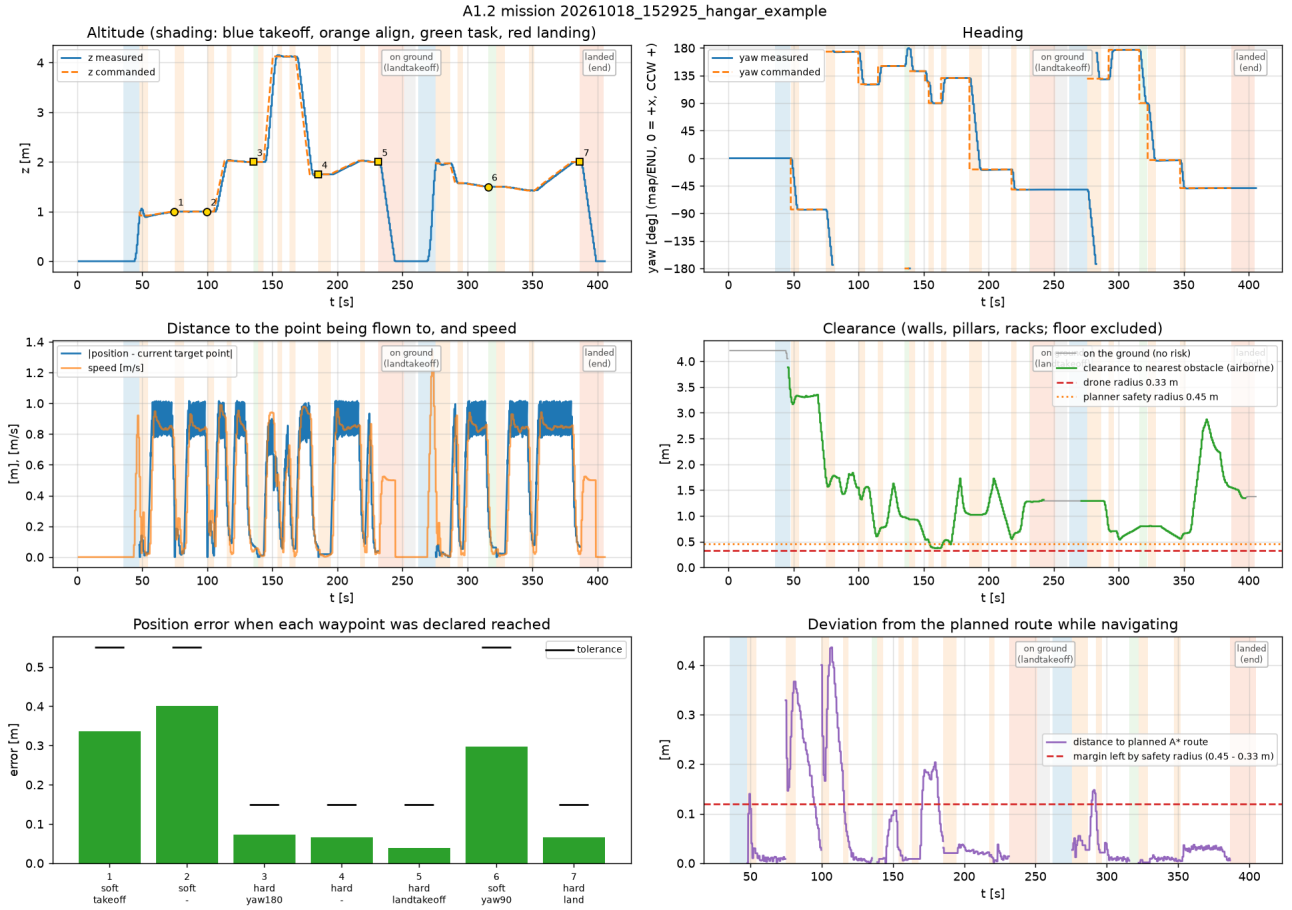
All missions ran unattended from start to disarm, with `safety_radius` 0.45 m. The recording is in `results/A1_2/safety_0.45_20261018/`.

Mission	Points	Result	Hard error	Soft error	Min clearance
`hangar_example.csv` (recorded)	7	DONE, disarmed	3.9–7.3 cm	0.30–0.40 m	0.37 m
`test_east_aisle.csv`	8	DONE, disarmed	2.9–7.0 cm	0.07–0.34 m	0.61 m
`test_over_racks.csv`	7	DONE, disarmed	5.0–7.6 cm	0.30–0.35 m	0.38 m

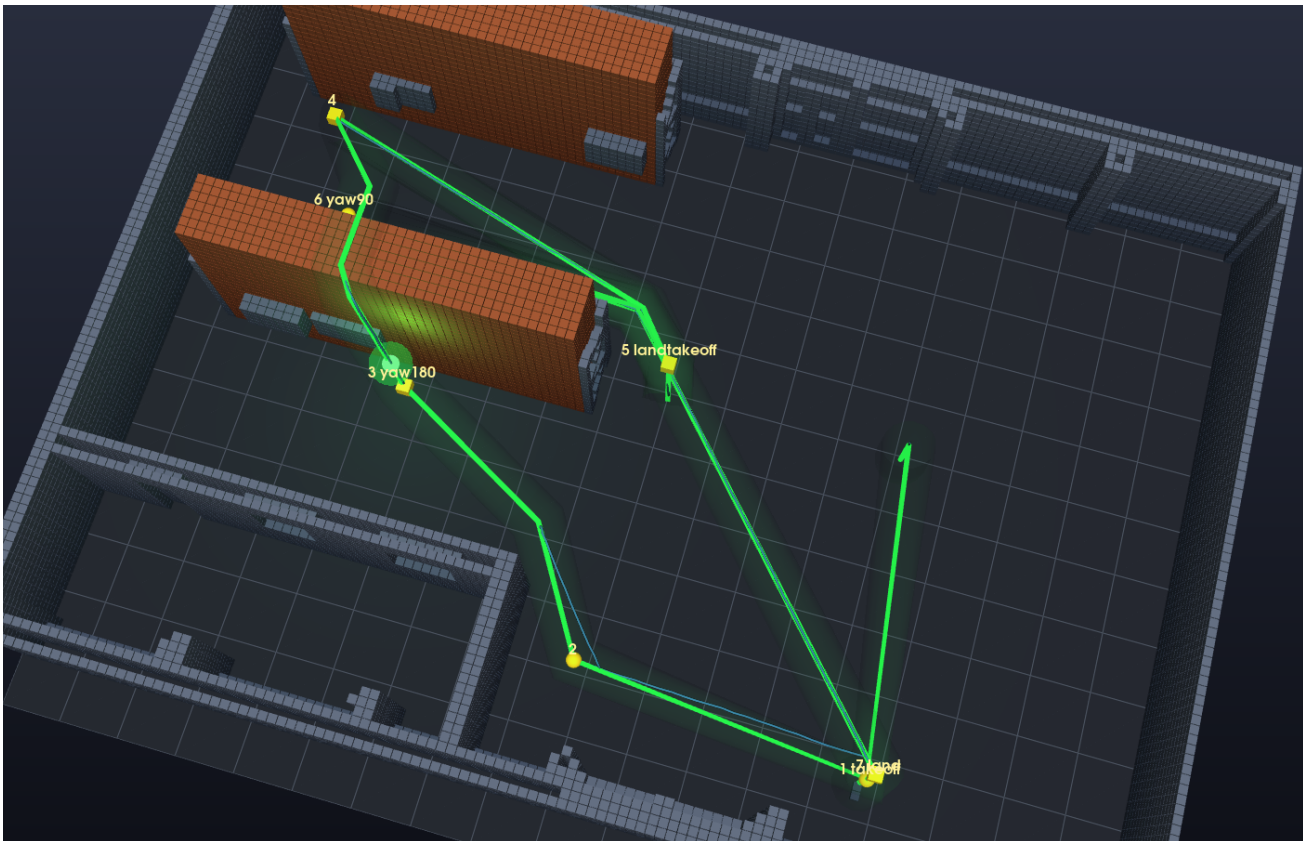
`hangar_example` took 404 s at a simulation speed of ~0.3; A* routed the leg to waypoint 4 over a rack (z ≈ 4.1 m).

Comparison with `safety_radius` 0.30 m (`results/A1_2/safety_0.30_20261019/`): the same mission ended DONE with every waypoint within tolerance (hard 5.1–8.2 cm, soft 0.31–0.42 m), but on the descent from the rack towards waypoint 5 the drone sank faster than it moved sideways and came within **0.20 m** of the rack edge, less than the 0.33 m drone radius (the propellers would hit the rack). That is why we use 0.45 m.





Commanded lines are blank during takeoff and landing on purpose: ArduCopter flies NAV_TAKEOFF / LAND itself and the executor sends no position setpoints then (a setpoint would cancel the takeoff). Route deviation is only defined while navigating.



Gaps in the plots are not missing data: during takeoff, landing and on the ground the executor deliberately sends no setpoint.

Known limitations

- **Small margin over the rack:** the smallest clearance is a few cm above the drone radius (voxel rounding in A1.1; on a descent the drone sinks ~0.15 m below the route). A larger `safety_radius` adds margin but blocks mission points near obstacles.
- **After a soft waypoint** the next leg starts at the waypoint centre, so for the first metre the drone can be up to 0.4 m from the drawn route (in open space).
- **Short stops at sharp corners** (> 20°) to turn the nose.
- **Gazebo must not be paused:** SITL runs in lock-step with Gazebo; ⏸ freezes ArduPilot and the mission ends in ERROR.